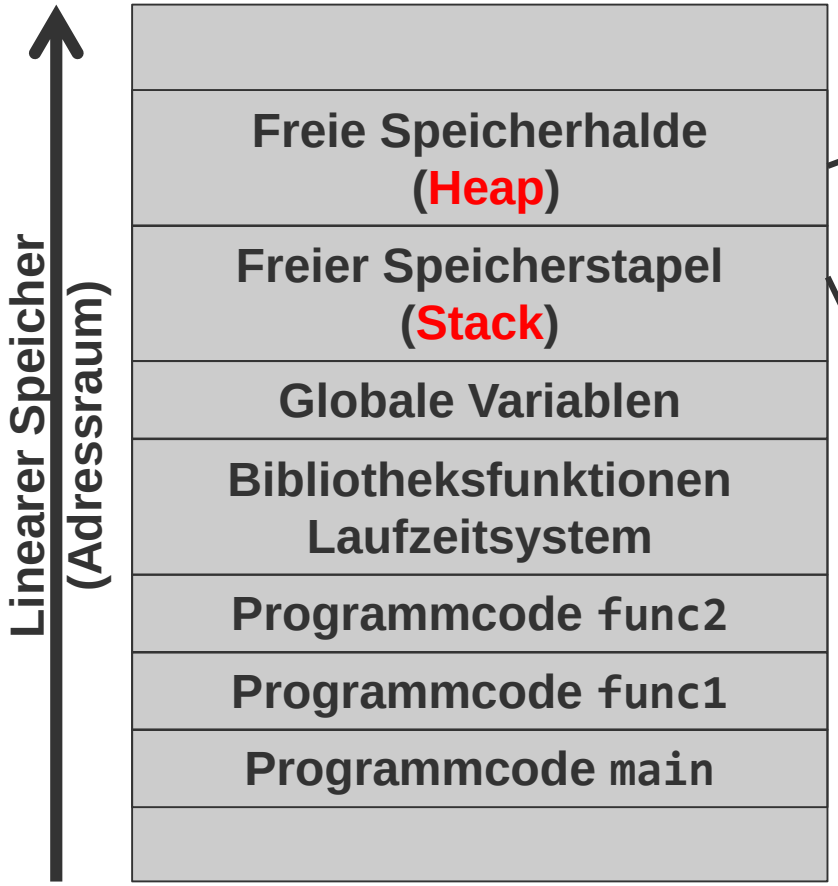
Speicher und Adressraum



Speicher und Adressraum

- Die Halde (Heap) ist ein Speicherbereich, aus dem zur Laufzeit eines Programms zusammenhängende Speicherabschnitte angefordert und in beliebiger Reihenfolge wieder freigegeben werden können.

- Der Stapel (Stack) ist eine Speicherbereich, auf den Elemente nur `oben´ gespeichert und auch nur von dort wieder gelesen werden. Elemente werden übereinander gestapelt und in umgekehrter Reihenfolge vom Stapel genommen.
  Dies wird auch Last In–First Out Prinzip (LIFO) genannt.

Speicher und Adressraum

- Verwendung der Halde

```
int[][] a;
a = new int[100][];          // Feld von 100 Verweisen

for (int i = 0; i < 100; i++)
    // 100 int-Felder mit Längen
    // von 1 bis 100
    a[i] = new int[i+1];
```

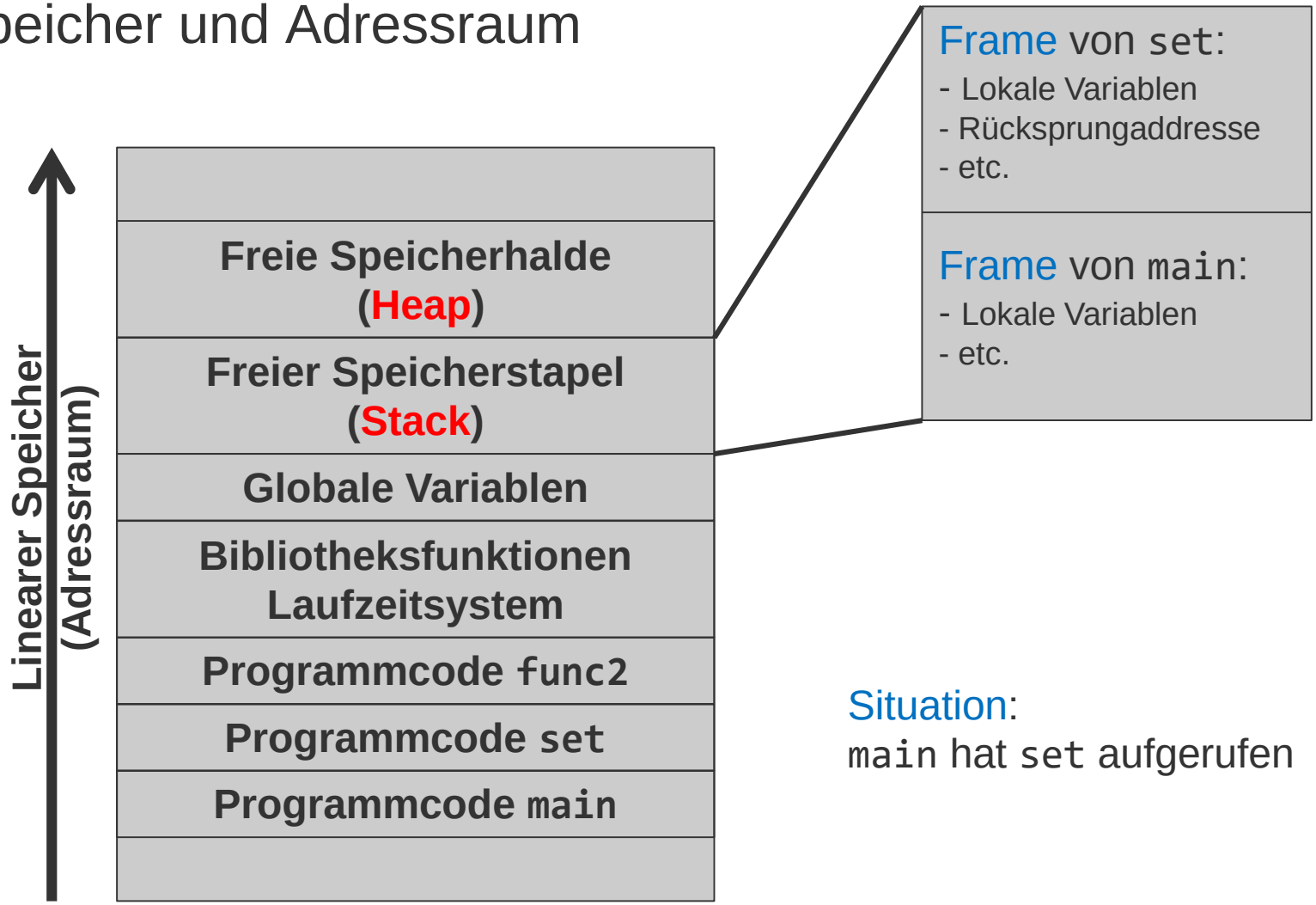
- Jedes dynamisch allokierte Feld wird auf der Halde angelegt
  - Egal ob die Allokation im Hauptprogramm oder in einer Funktion stattfindet
  - Für den Programmierer erst einmal nicht sichtbar

Speicher und Adressraum

- Verwendung des Stacks – Funktionsaufrufe

```
public static void set(int x)  {
    x = 4;
    return;
}

public static void main(String[] args) {
    int x = 2;
    set(x);

    write(x);
}
```

Speicher und Adressraum



Situation:
`main` hat `set` aufgerufen

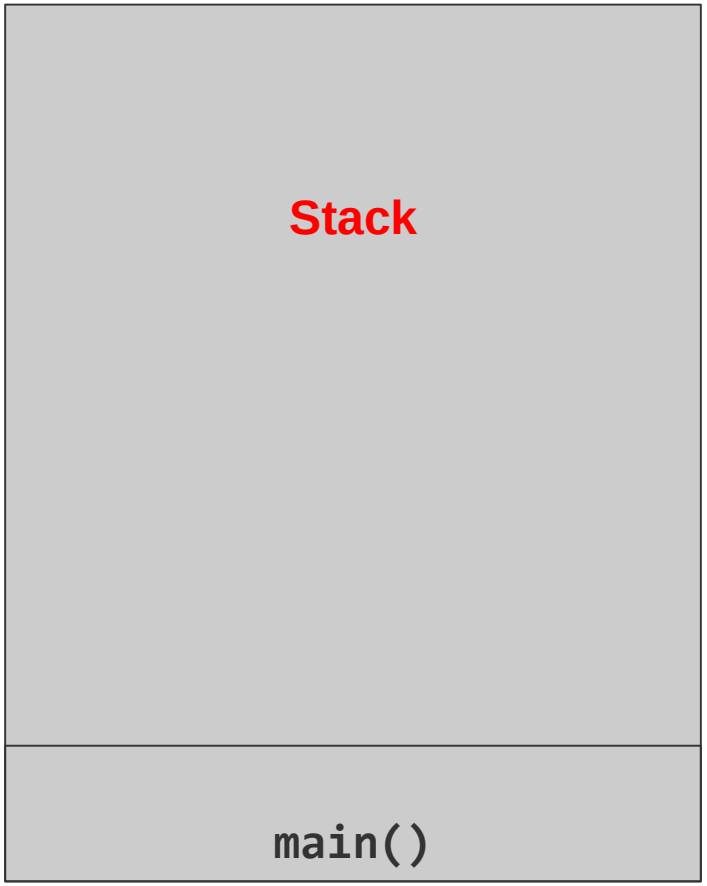
Speicher und Adressraum

- Verwendung des Stacks – Funktionsaufrufe
  - Beim Aufruf von main und den Funktionen/Prozeduren werden die jeweils lokalen Variablen (und evtl. die Rücksprungadresse) als sog. Frames auf den Stack gespeichert
  - Während der Programmausführung ist jeweils nur der oberste Frame `sichtbar´, und die als global definierten Variablen
  - Bei der Beendigung einer Funktion/Prozedur (Rücksprung in das Hauptprogramm) wird der zur Funktion/Prozedur gehörende Stack gelöscht

Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

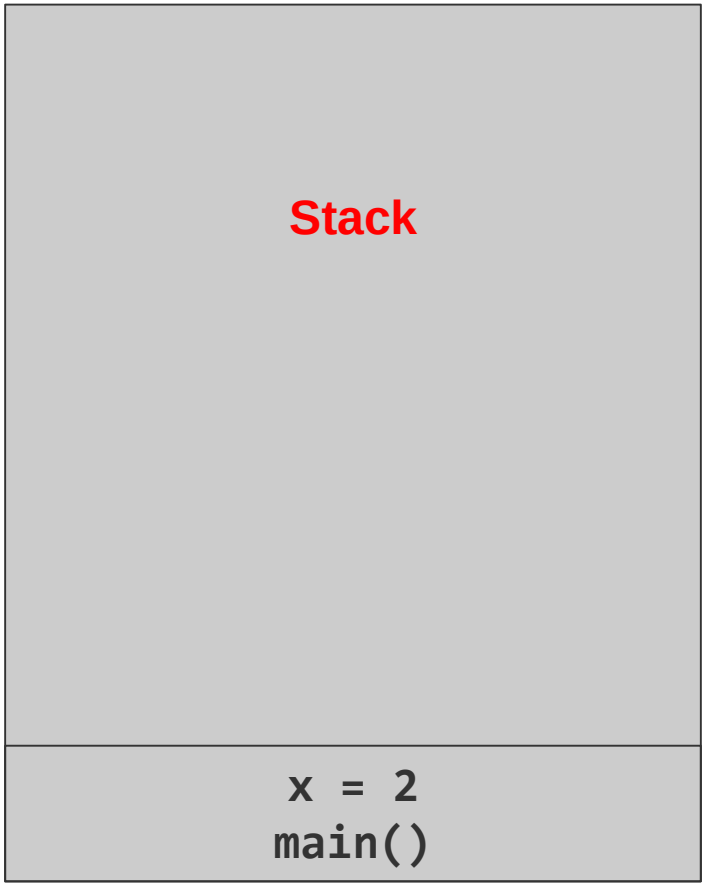
```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```



Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

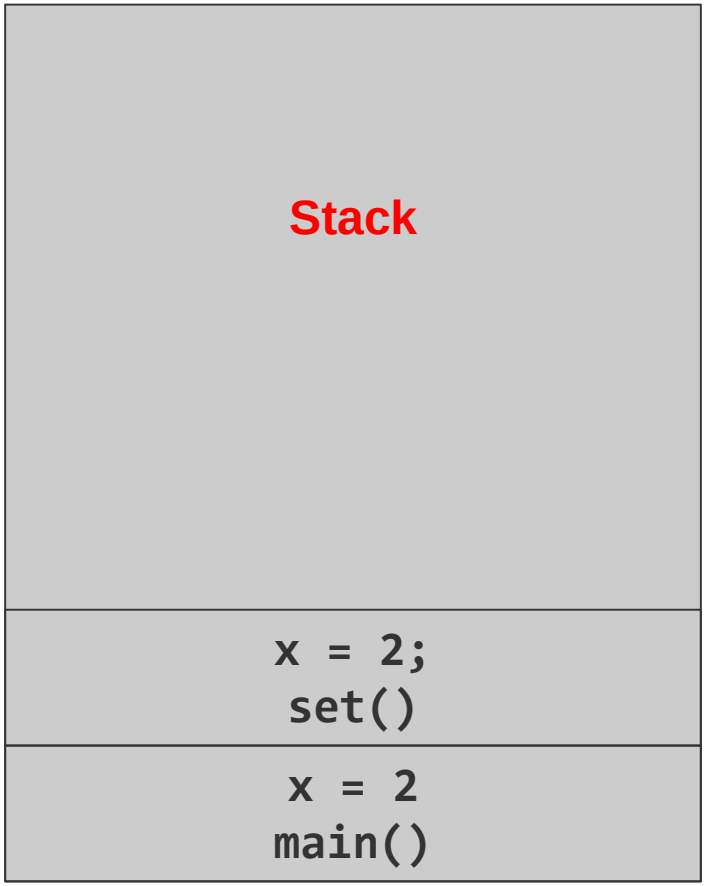


Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

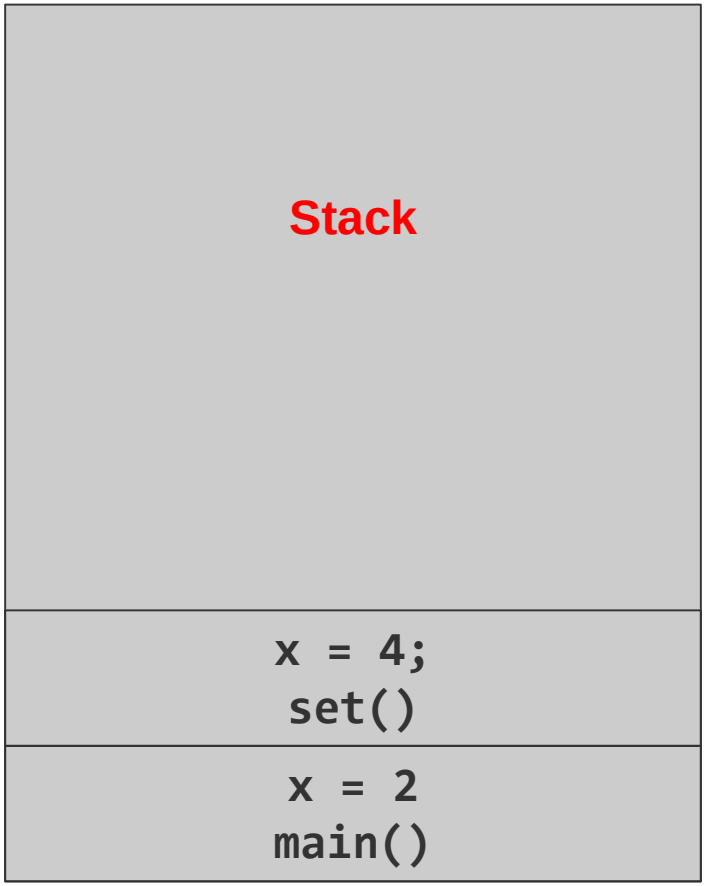
Kopieren des Wertes von x nach lokales x in `set`



Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

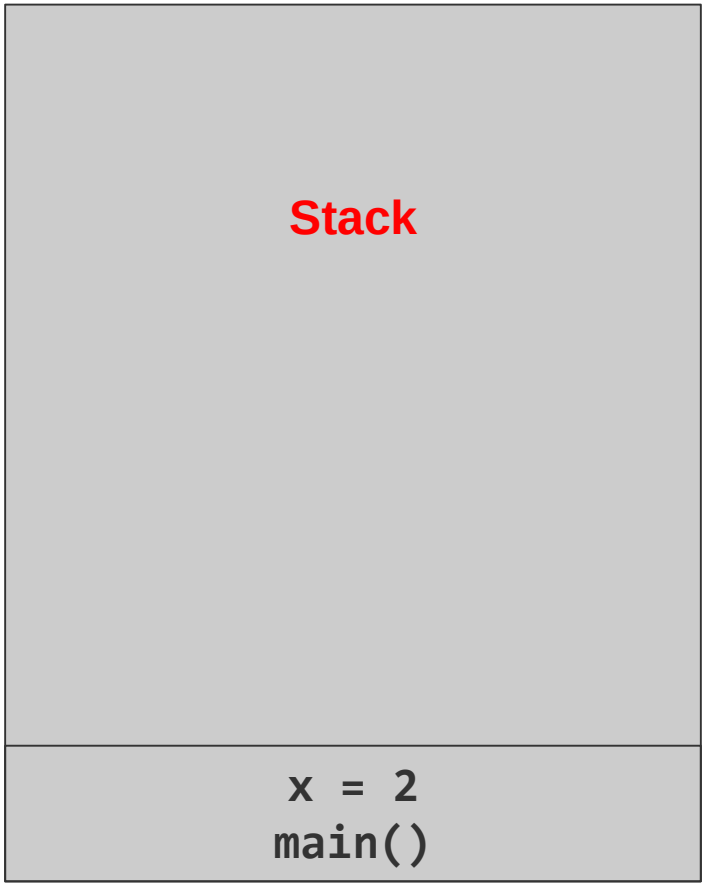
```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```



Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

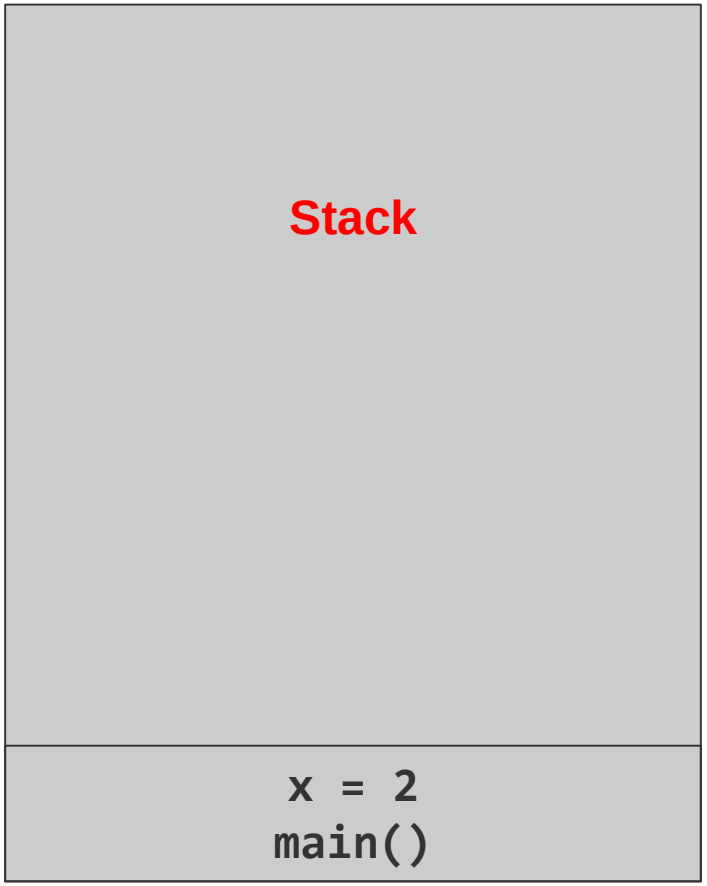
```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```



Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

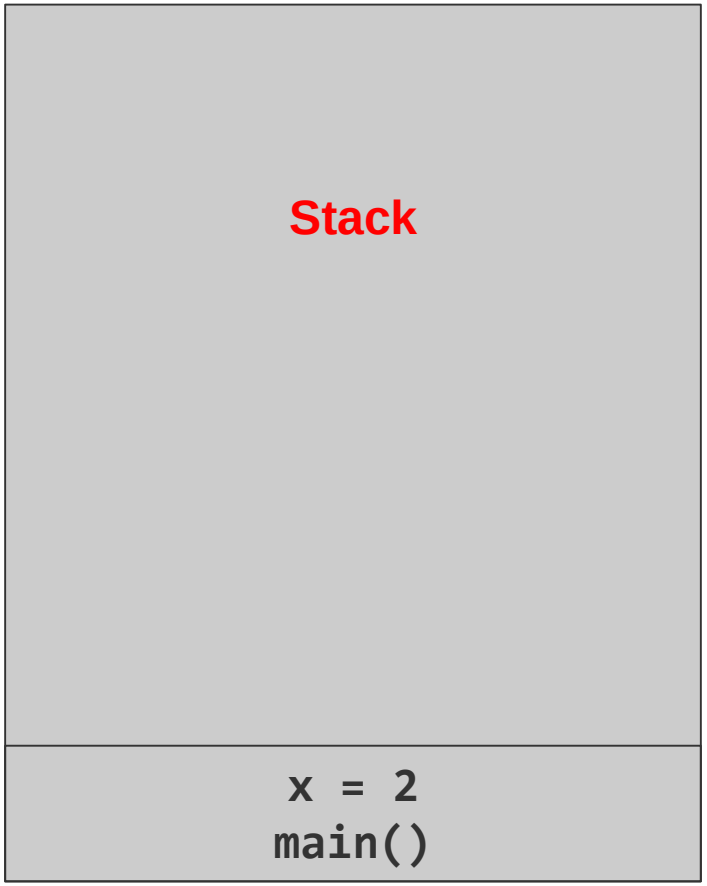


Speicher und Adressraum

- Parameterübergabe an Funktionen in Java

```
public static void set(int x)  {
      x = 4;
      return;

}

public static void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

- Diese Art der Parameterübergabe nennt man **Call-by-Value**

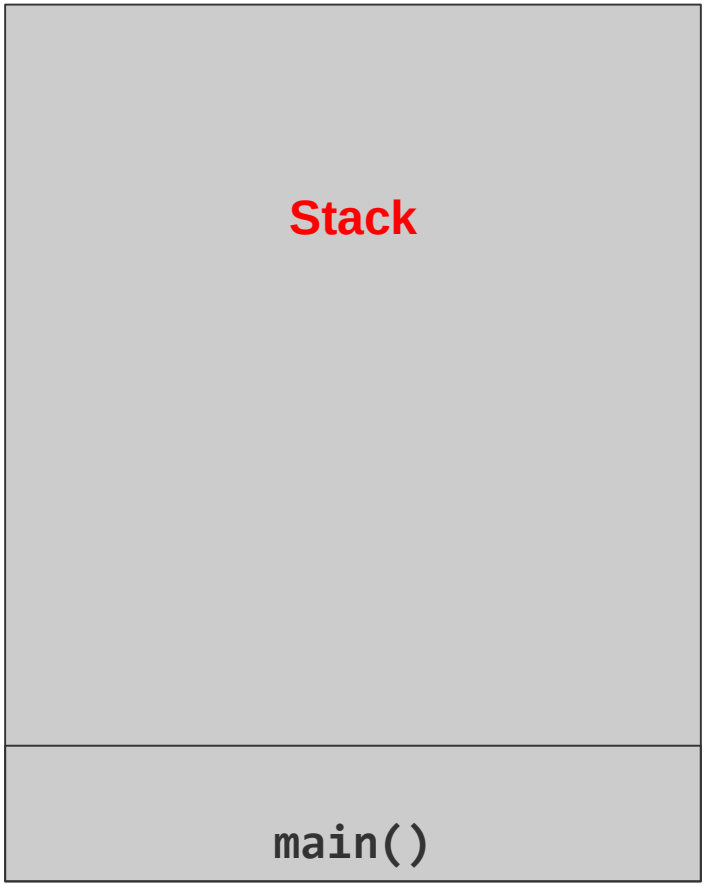


Speicher und Adressraum

- Parameterübergabe mittels **Call-by-Reference**

```
void set(int &x)  {
      x = 4;
      return;

}

void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

- Die Funktion erhält die Adresse (&) von x und nicht dessen Wert

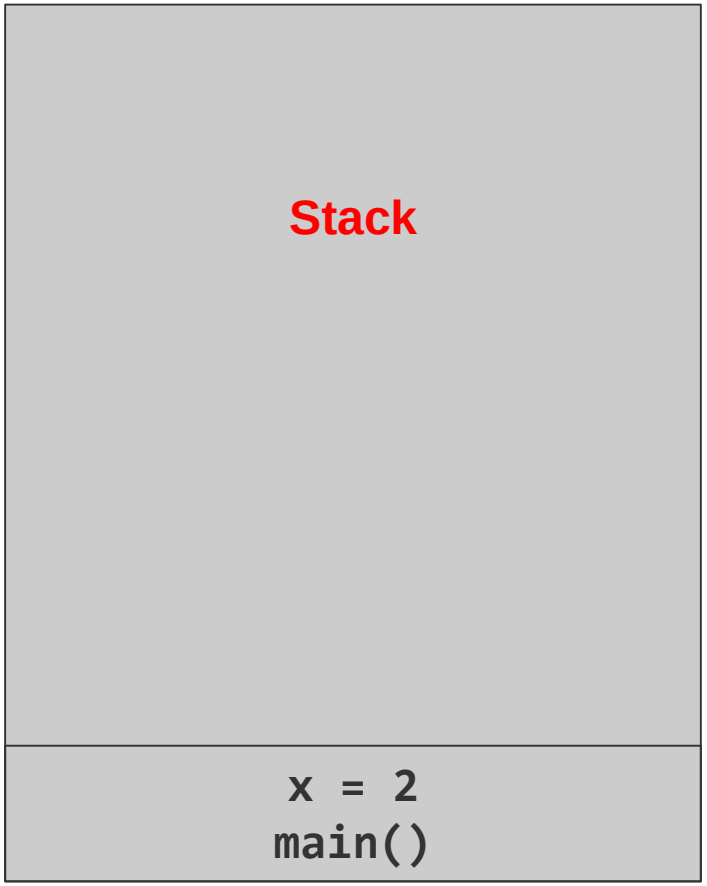


Speicher und Adressraum

- Parameterübergabe mittels **Call-by-Reference**

```
void set(int &x)  {
      x = 4;
      return;

}

void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

- Die Funktion erhält die Adresse (&) von x und nicht dessen Wert

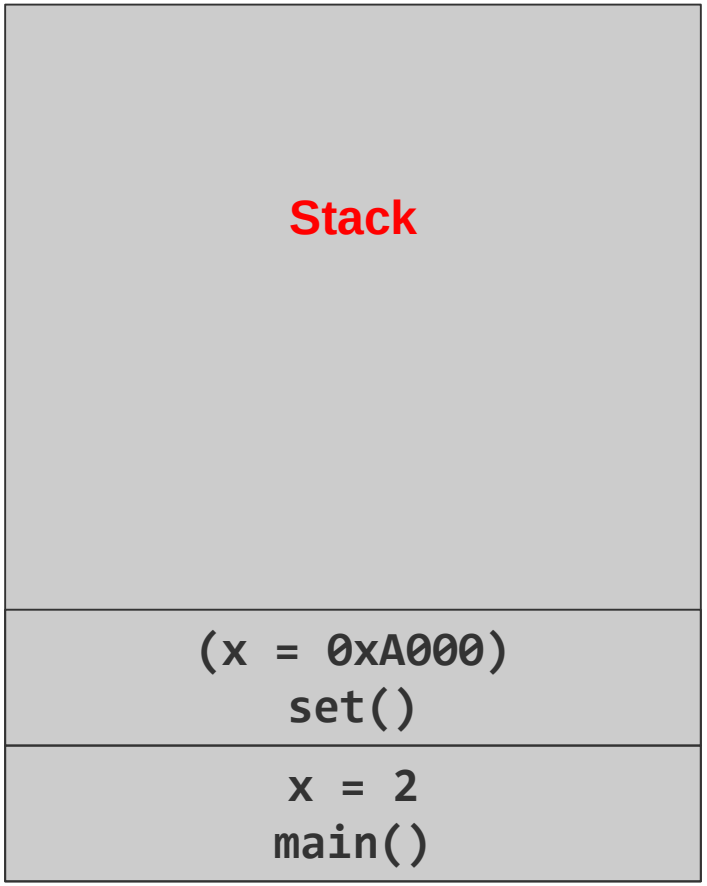


Speicher und Adressraum

- Parameterübergabe mittels **Call-by-Reference**

```
void set(int &x)  {
      x = 4;
      return;

}

void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```

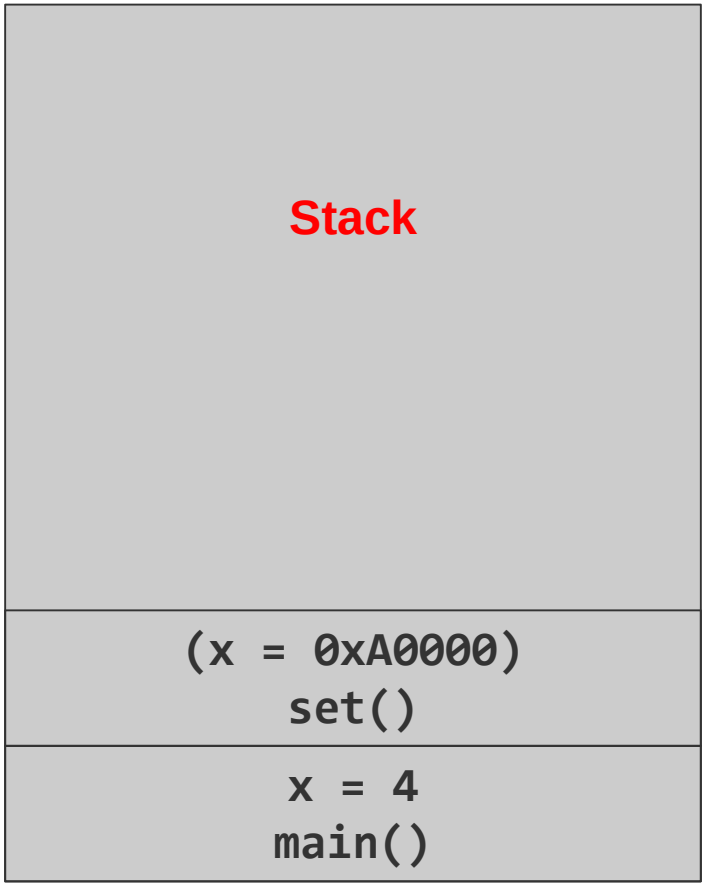
Kopieren der Adresse von x nach lokales x in `set`



Speicher und Adressraum

- Parameterübergabe mittels **Call-by-Reference**

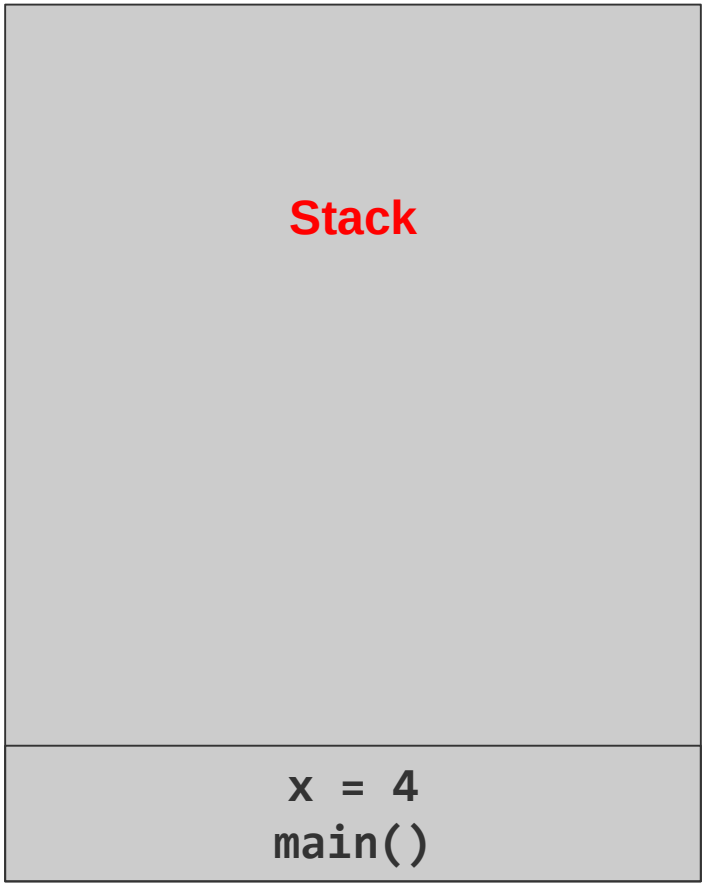
```
void set(int &x)  {
      x = 4;
      return;

}

void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```



Speicher und Adressraum

- Parameterübergabe mittels **Call-by-Reference**

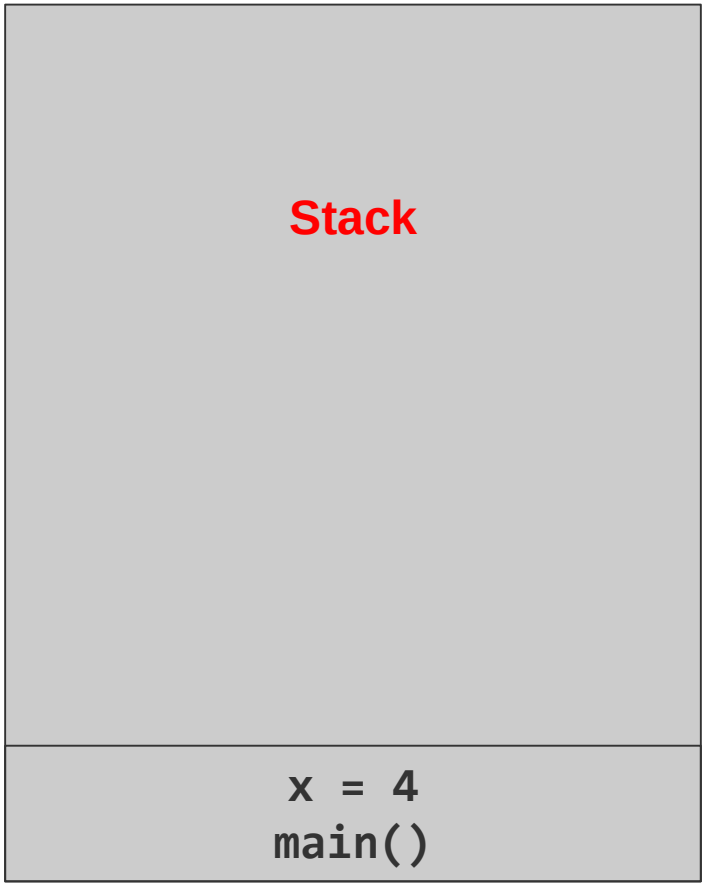
```
void set(int &x)  {
      x = 4;
      return;

}

void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```



Speicher und Adressraum

- Parameterübergabe mittels **Call-by-Reference**

```
void set(int &x)  {
      x = 4;
      return;

}

void main(String[] args) {
         int x = 2;
         set(x);

         write(x);
}
```



Speicher und Adressraum

- Java kennt keinen Adress-Operator &
- Für primitive Datentypen wird automatisch ein Wertparameter verwendet, der beim Aufruf einer Funktion kopiert wird
  → **Call-by-Value**
- Für Objekte (siehe später) wird die Objektreferenz als Wertparameter durch Kopie übergeben
  → dies ist nicht **Call-by-Reference**, da die Referenz eine lokale Kopie der aufrufenden Instanz ist

Speicher und Adressraum

- Java Beispiel

```
void set(int [] a)  {
      a[10] = 100; // Änderung des 10-ten Elements in a

      int [] b = new int [100];
      for(int i = 0; i < 100; i++)
          b[i] = 1;

      a = b;       // keine Änderung auf a in main !!!
      return;

}

void main(String[] args) {
         int [] a = new int[100];
         set(a);

         ...
}
```

Speicher und Adressraum

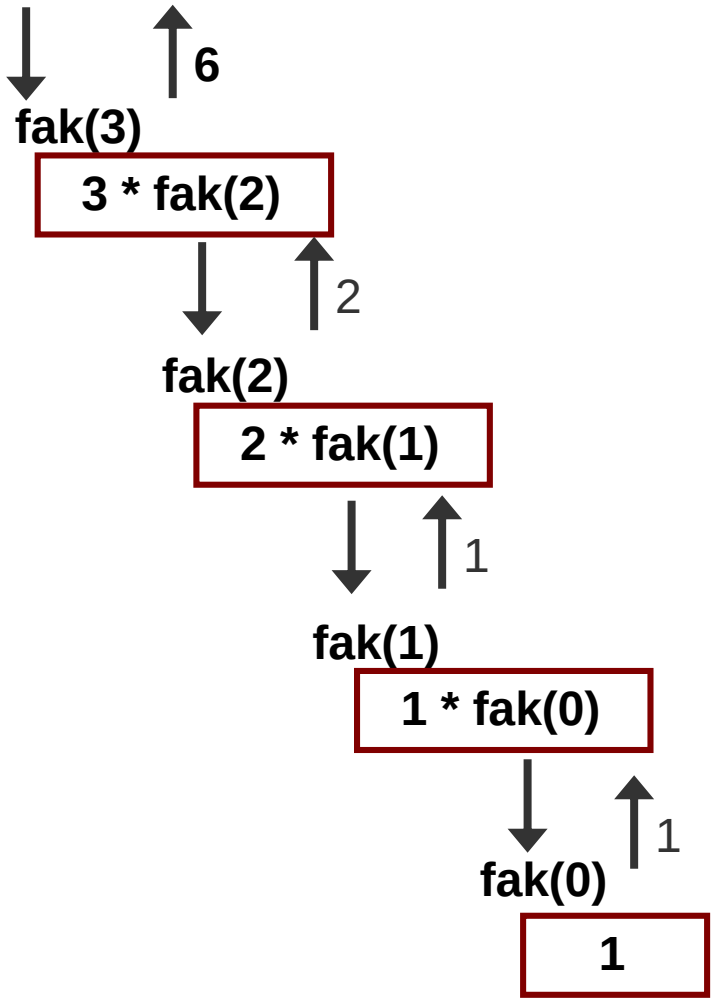
Rekursive Funktionen

- Verwendung des Stacks –
  Rekursion am Beispiel der Fakultätsberechnung

```
public static int fak(int n)  {
    if (n > 0)
          return n * fak(n-1);
    else
          return 1;
 }
```

- Definition einer Funktion, die sich selbst aufruft, um den Funktionswert zu berechnen.

Fakultätsberechnung

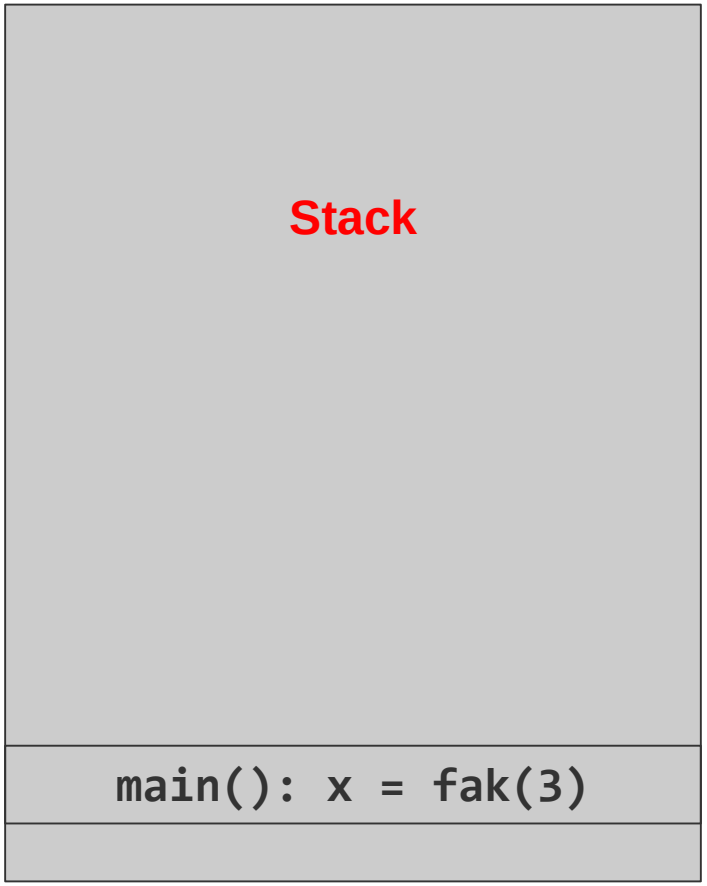



```
public static int fak(int n)  {

    if (n > 0)
        return n * fak(n-1);
    else
        return 1;

}
```

Rekursive Funktionen

- Aufbau des Frame-Stacks

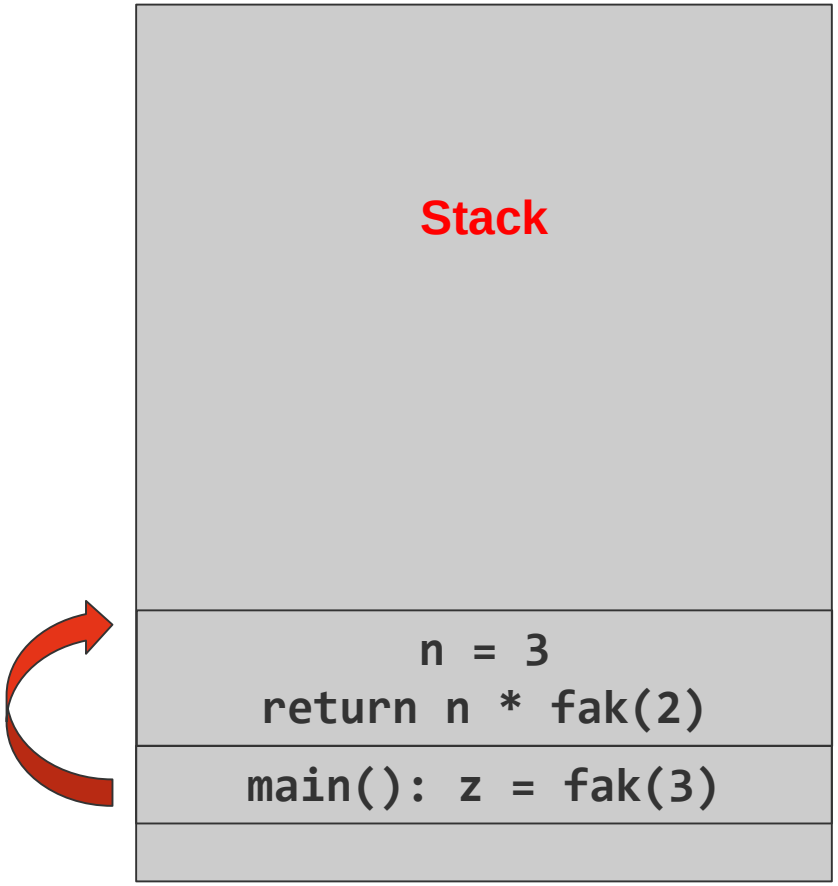
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Aufbau des Frame-Stacks

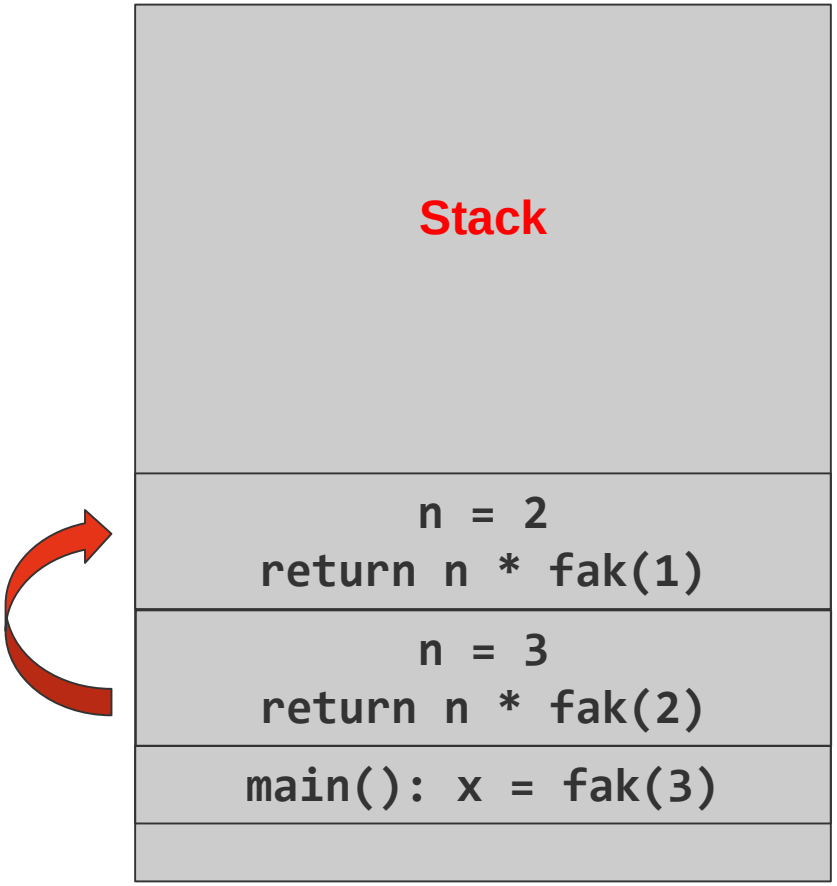
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Aufbau des Frame-Stacks

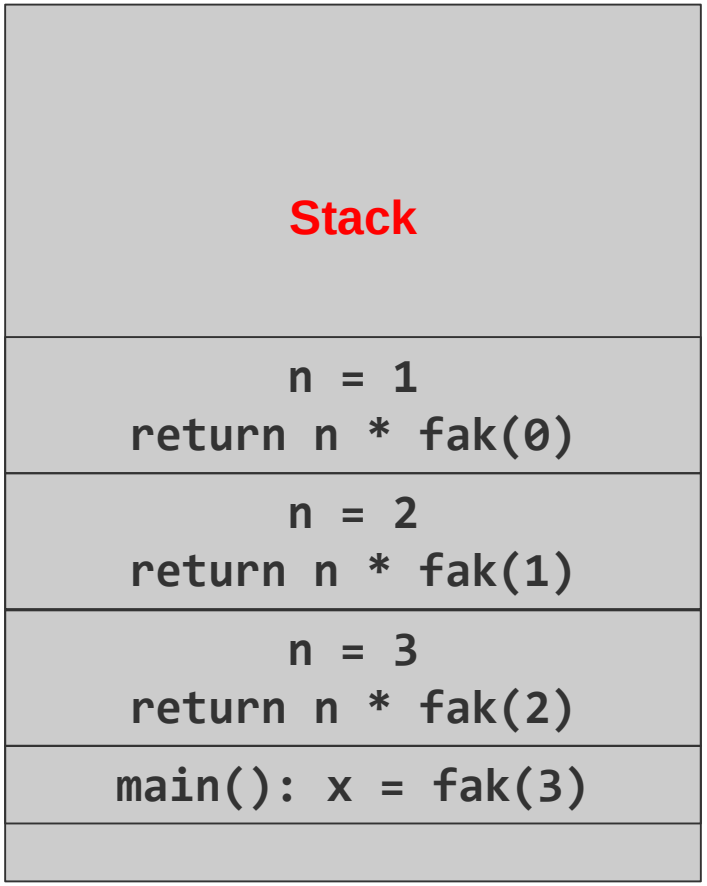
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Aufbau des Frame-Stacks

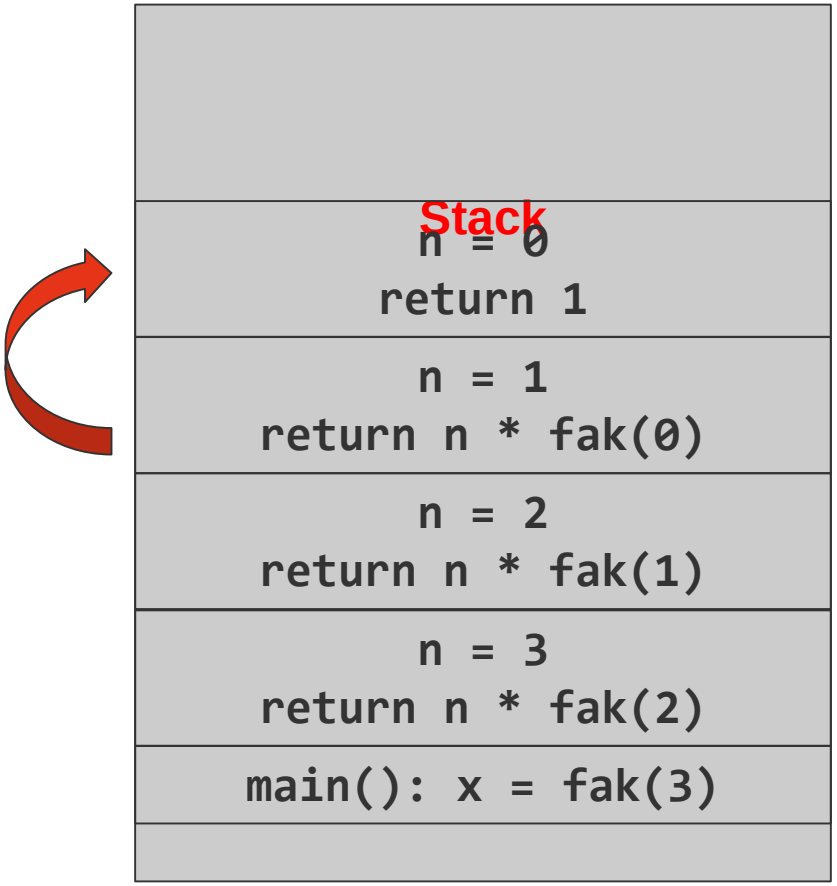
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Aufbau des Frame-Stacks

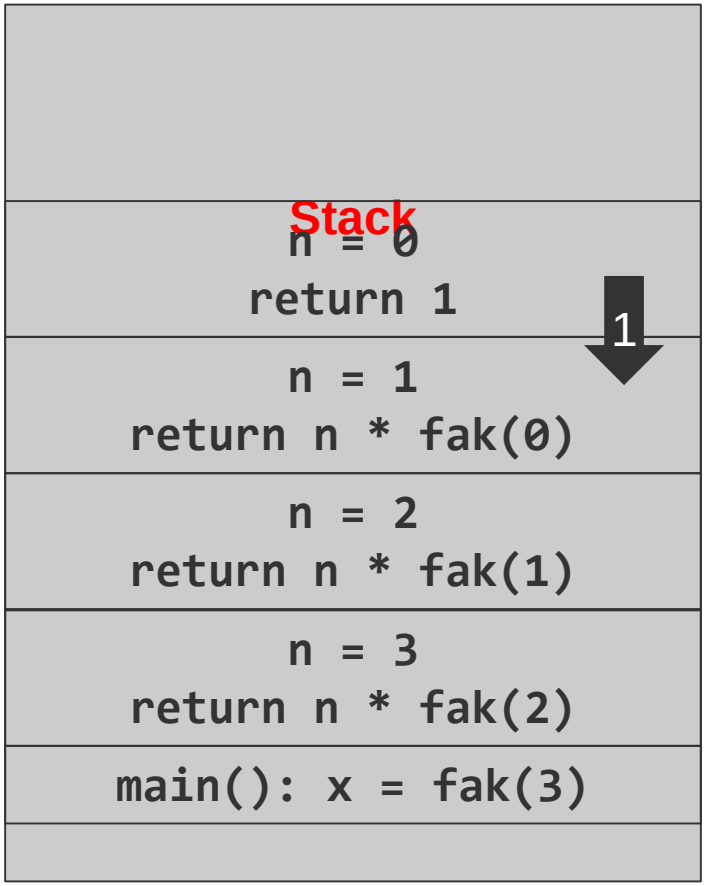
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Aufbau des Frame-Stacks

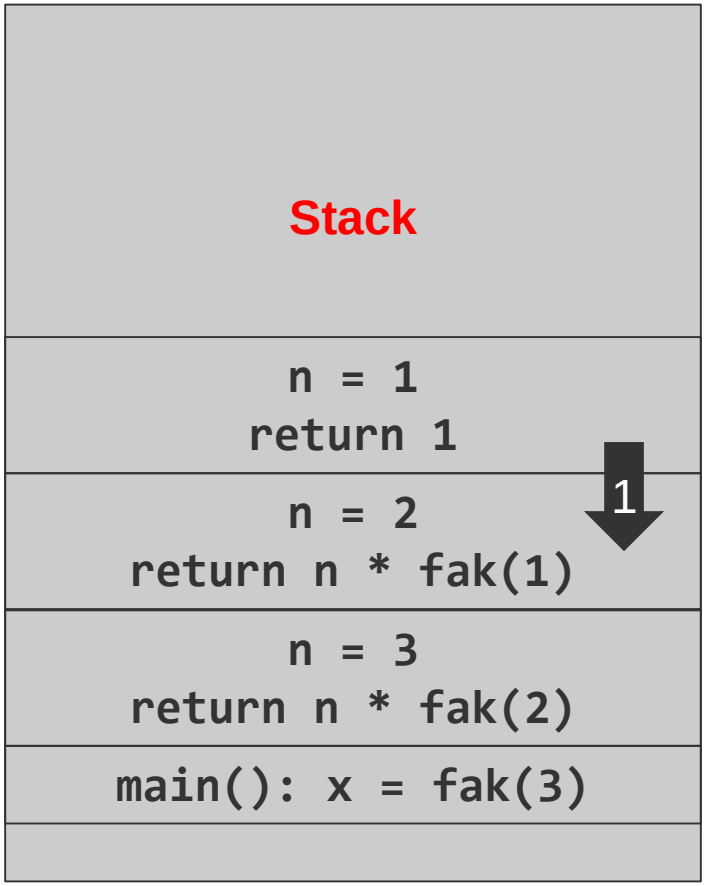
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Abbau des Frame-Stacks

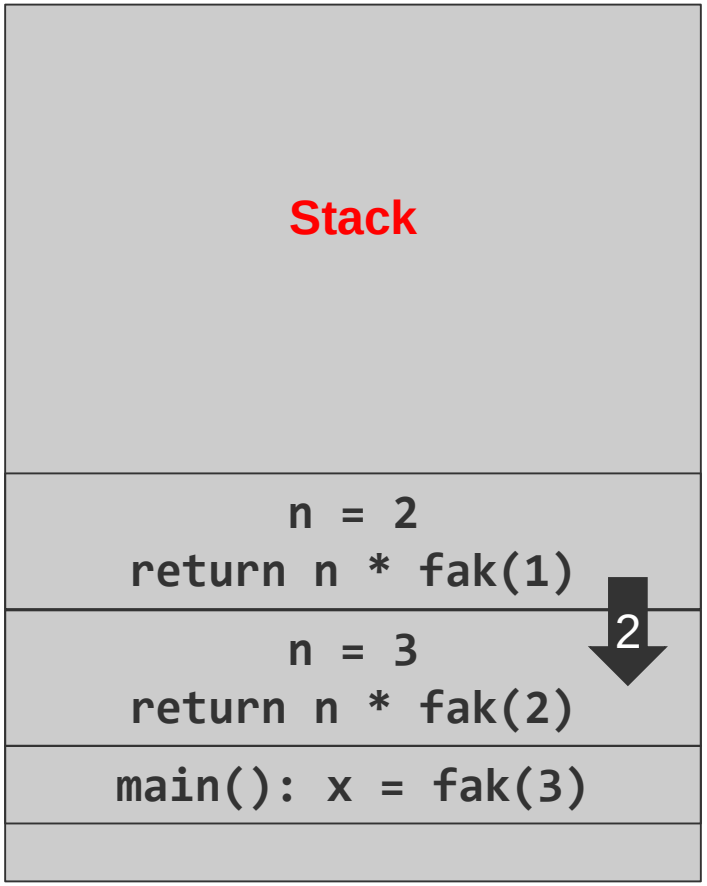
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Abbau des Frame-Stacks

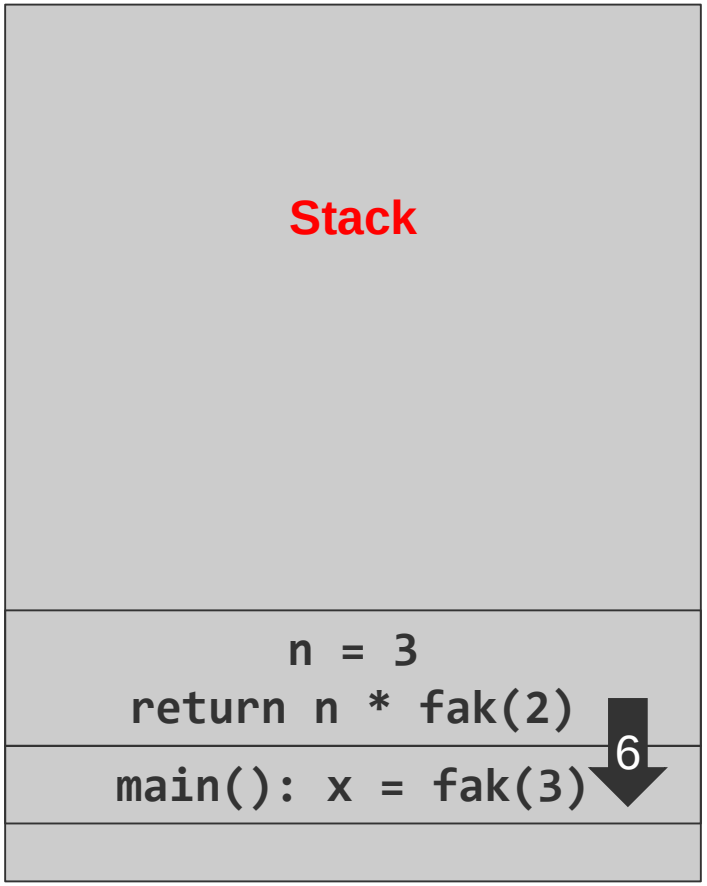
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Abbau des Frame-Stacks

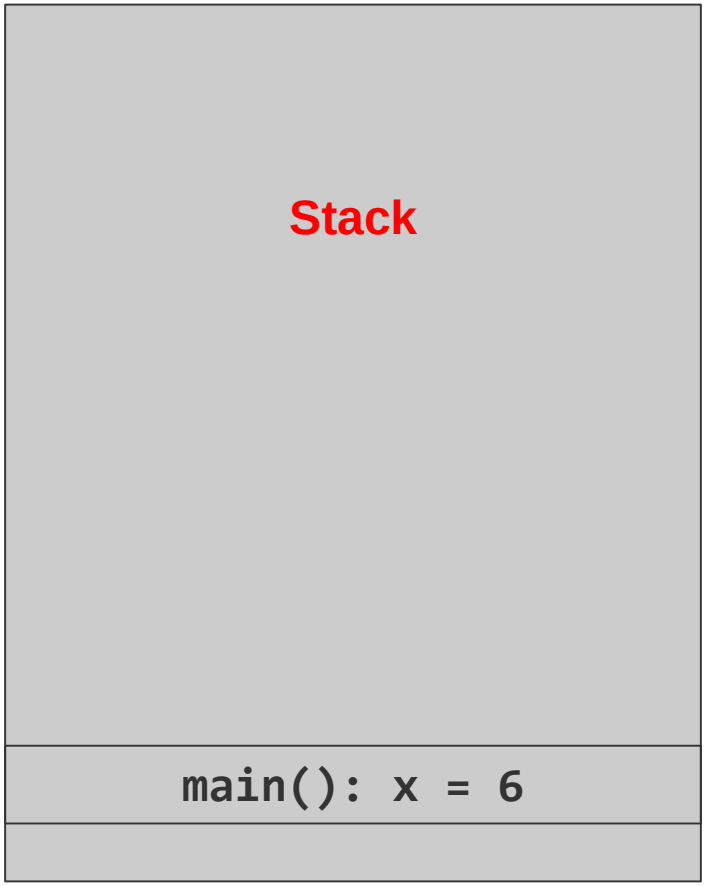
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursive Funktionen

- Abbau des Frame-Stacks

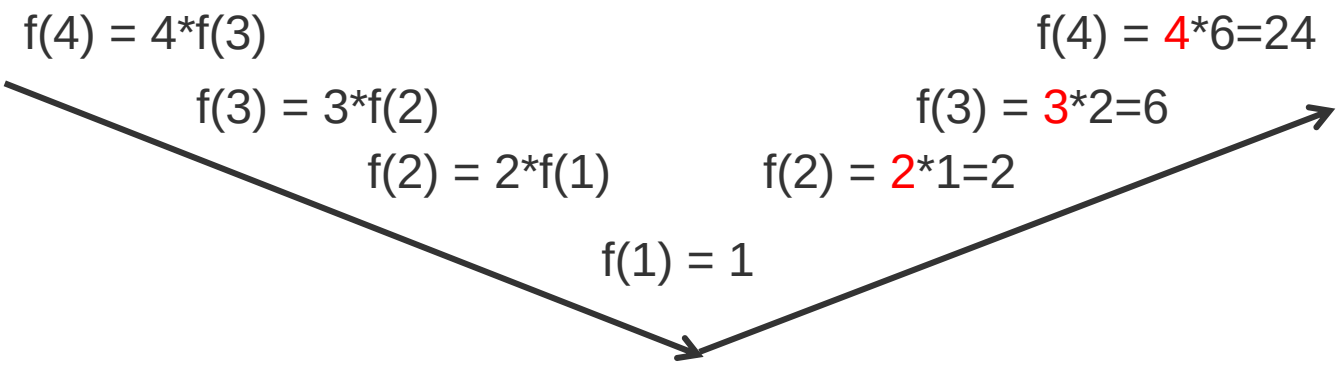
```
public static int fak(int n)  {

   if (n > 0)
      return n * fak(n-1);
   else
      return 1;

}

public static void main(String[] args) {
   int x;
   x = fak(3);

   write(x);
}
```



Rekursion

- (linear) rekursive Fakultätsberechnung (ein rekursiver Aufruf)

```
public static int f(int n)  {
    if (n > 0)
          return n * f(n-1);
    else
          return 1;
}
```



```
int f(int akku, int n) {
   if (n == 0)
      return akku;
   else
      return h(n*akku,n-1);
}

int fak = h(1,n) ;
```

Rekursion

- Beobachtung:
  In der rekursiven Berechnung der Fakultät wird nach der Rekursion noch eine Multiplikation mit dem Argument n berechnet
  - Es müssen Zwischenergebnisse auf dem Stack gespeichert werden und
  - Die Funktion kann am Ende der Rekursion nicht direkt terminieren
- Eine Rekursion heißt endrekursiv, wenn der Aufruf der Rekursion die letzte Aktion zur Berechnung der rekursiven Funktion ist
  - z.B. konkret: return *rekFunc(…);*
  - Endrekursive Funktionen zeigen ein besseres Verhalten hinsichtlich des Speicherbedarfs
- Wie kann man eine nicht endrekursive Funktion endrekursiv machen?

```
int GGTRec(int a, int b) {
   if(b==0)
      return a;
   else
      return GGTRec(b,a%b);
}
```

Rekursion

Hinweis:
% ist der Modulo-Operator.
a % b liefert den Rest der Ganzahldivision a / b

- Iteration anstelle von Rekursion

Die Endrekursion lässt sich immer durch eine n-fache while-Schleife berechnen

- Negieren der Abbruchbedingung der Rekursion und Verwenden als Bedingung der while-Schleife
- Der Rest des Funktionsrumpfs wird zum Rumpf der while-Schleife.
- Die endrekursiven Aufrufe (return *rekFunc(…);*) werden zu Zuweisungen an die Parameter

```
int GGTRec(int a, int b) {
   if (b==0)
      return a;
   else
      return GGTRec(b,a%b);
}
```

```
int GGTI(int a, int b) {
   while (b > 0) {
      int temp = b;
      b = a % b;
      a = temp;
   }
   return a;
}
```

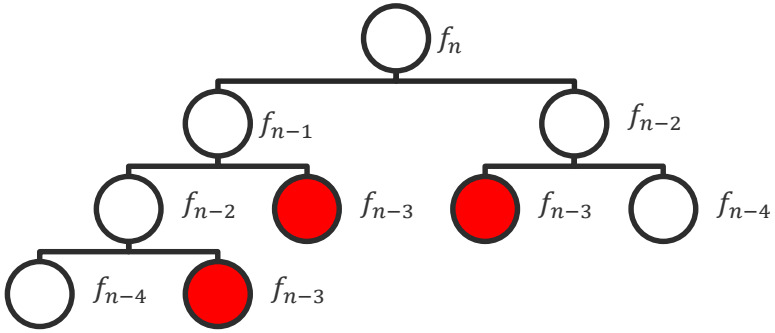
Rekursion

- Ein Beispiel für eine nichtlineare Rekursion (mehrfache rekursive Aufrufe in einer Funktion)

  Die Berechnung der Fibonacci Zahlen: 1, 1, 2, 3, 5, 8, 13, 21, 34, ... mit der Rekursionsgleichung

  $$f_n \coloneqq f_{n-1} + f_{n-2} \text{ mit } f_1 \coloneqq f_2 \coloneqq 1$$

- Die Aufruffolge zeigt, dass viele Fibonacci Zahlen mehrfach berechnet werden und $\sim 2^n$ Auswertungen benötigt werden



Rekursion

- Auch nichtlinear rekursive Funktionen lassen sich mit while-Schleifen berechnen

- Iterative Berechnung der Fibonacci Zahlen

```
public static int fib(int n) {
    int last = 1;
    int butLast = 1;
    while(n >= 2) {
        int temp = last;
        last = last + butLast;
        butLast = temp;
        n--;
    }
    return last;
}
```

Endrekursive Variante

```
fib(int n, int a , int b) {
   if (n <= 1)
      return(a + b);
   return fib(n-1, b+a, a);
}

fibEndrekursiv(int n) {
   return fib(n-1,1,0);
}
```

Zusatz

- Die Operatoren && und || in Java

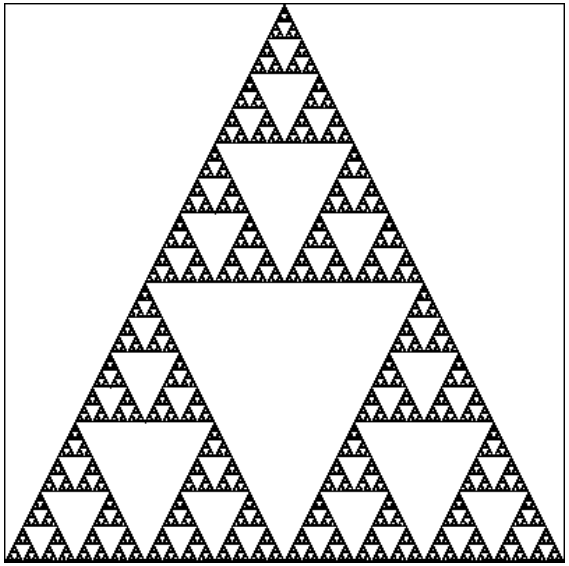
- In Java, && ist der logische AND Operator. Er wird verwendet, um zwei boolesche Ausdrücke miteinander zu kombinieren. Er liefert true (1) nur dann, wenn beide Ausdrücke true sind. Ist einer der beiden Ausdrücke false, dann ist das Resultat false (0).

- In Java, || ist der logische OR Operator. Er wird verwendet, um zwei boolesche Ausdrücke miteinander zu kombinieren. Er liefert true, wenn mindestens einer der beiden Ausdrücke true ist. Nur wenn beide Ausdrücke false sind, ist auch das Resultat false.

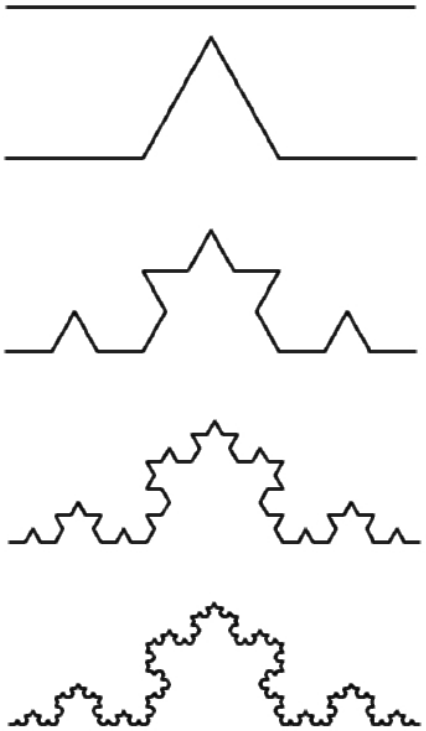
Rekursion

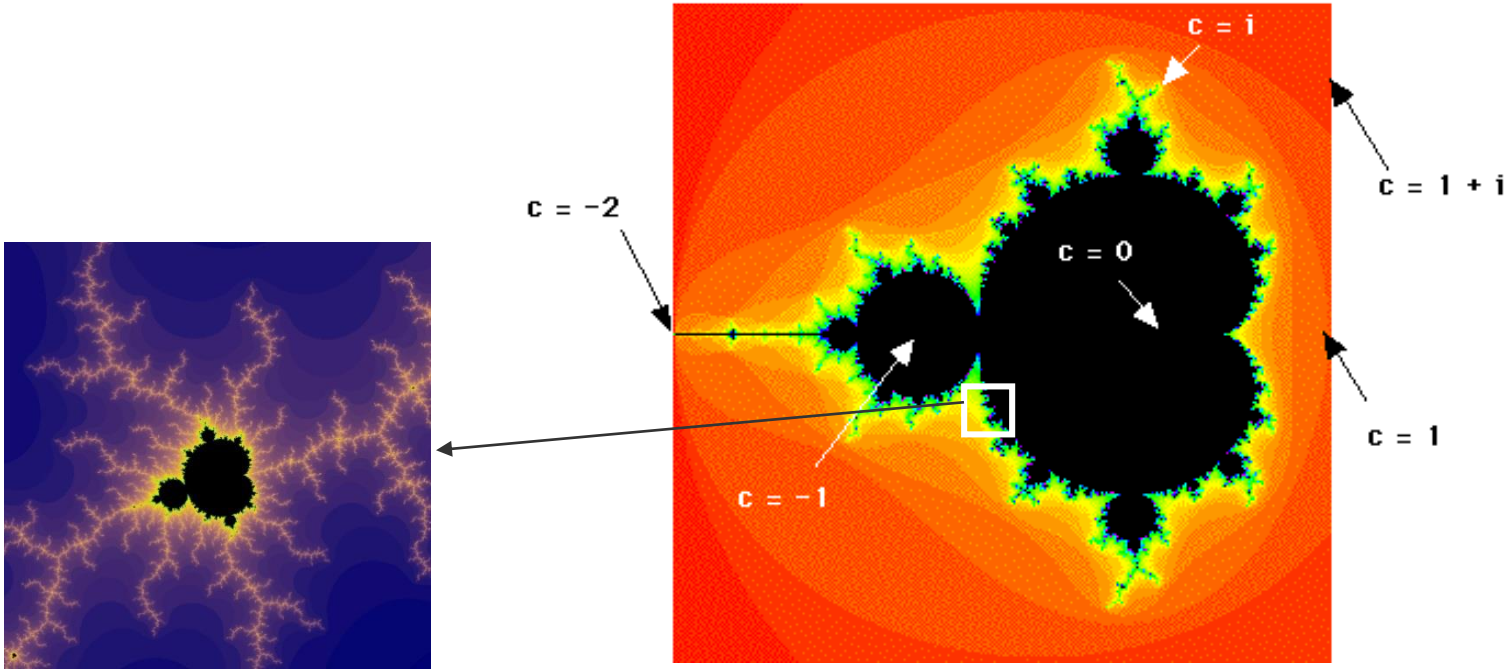
- Weitere Beispiele für rekursive Strukturen - Fraktale

Sierpinski-Dreieck



Koch-Kurve



Rekursion

- Rekursion in der Natur: die Mandelbrot Menge
  - Rekursive Auswertung der Rekursionsgleichung für initiale Punkte c in der komplexen Zahlenebene ($z_i = 0$)
  - $z_{i+1} = z_i^2 + c$ (Färbung der Punkte zeigt Stärke des Wachstums von z_i)